

## Demonstrating Metastability Without Synchronization

### Introduction

Metastability is a phenomenon that occurs when a flip-flop samples an asynchronous (unsynchronized) input signal whose transitions are close to the clock's rising edge. In such cases, the flip-flop output cannot decide on a stable logic level (0 or 1) instantly, leading to unpredictable or unstable states, sometimes observed as glitches in simulation and, in real hardware, as increased risk of failures.

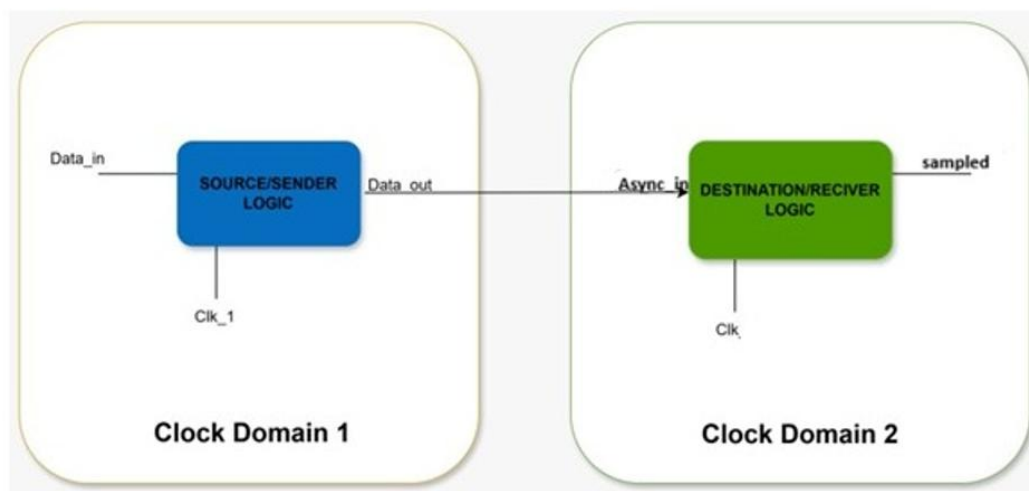
This lab demonstrates how directly sampling an asynchronous input without synchronization can lead to metastable behavior, using Verilator and GTKWave to visualize the simulation.

### Concept:

If an input changes close to the active clock edge, the flip-flop may not resolve to a reliable logic state immediately. Instead:

- Its output might oscillate or produce short glitches.
- Simulation waveforms may show undefined or brief pulses.
- In real circuits, metastability shortens the setup/hold timing margins and can cause downstream logic errors.

### Block Diagram:



**Fig: Metastability Without Synchronization**

## Application:

Understanding metastability and the dangers of directly sampling asynchronous inputs is crucial in:

- Digital system interfacing (crossing clock domains)
- Data acquisition from external devices
- Safe design of reliable communication protocols
- Design of synchronizer circuits for robust systems

## Procedure To Create Project

### Step 1: Set up Project Directory

1.1: Create a directory for your lab: `mkdir metastability_lab`

1.2: Open lab directory: `cd metastability_lab`

### Step 2: Create RTL Design File

2.1: Create design file: `gvim metastability.v`

// Module to sample async input directly without synchronization

```
module metastability (  
    input clk,           // Clock input  
    input async_in,      // Asynchronous input signal  
    output reg sampled //Sampled output (may go metastable)  
);  
    always @(posedge clk) begin  
        sampled <= async_in;  
    end  
endmodule
```

2.2 : Save as metastability.v using :wq

### Step 3: Create Testbench File

3.1: Create testbench file: `gvim metastability_tb.v`

// Testbench for metastability

```
module metastability_tb;
```

```
reg clk = 0;

reg async_in = 0;
wire sampled;
always #5 clk = ~clk;
// Device Under Test instantiation
metastability uut (
    .clk(clk),
    .async_in(async_in),
    .sampled(sampled)
);
initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, metastability_tb);
    #12 async_in = 1;    // Change async_in near clock edge
    #7  async_in = 0;
    #6  async_in = 1;
    #9  async_in = 0;
    #50 $finish;
end
endmodule
```

### 3.2: Save file as metastability\_tb

## Step 4: Create Simulation Script

### 4.1: Create run.sh

```
#!/bin/bash
```

# 1: Run Verilator to generate simulation files

```
verilator --binary -j 0 -Wall metastability.v \
metastability_tb.v --top metastability_tb --timing \
--CFLAGS "-std=c++20" --trace
```

# 2: Move into the object directory

```
cd obj_dir || { echo "obj_dir not found"; exit 1; }
```

# 3: Compile the testbench using make

```
make -f Vmetastability_tb.mk Vmetastability_tb || { echo  
"Compilation failed"; exit 1; }
```

# 4: Run the testbench simulation

```
./Vmetastability_tb || { echo "Simulation failed"; exit 1; }
```

# 5: View the waveform using GTKWave

```
gtkwave dump.vcd
```

4.2 : Save the script file.

4.3: Make the script executable:

```
chmod +x run.sh
```

## Step 5: Run And Observe the Output

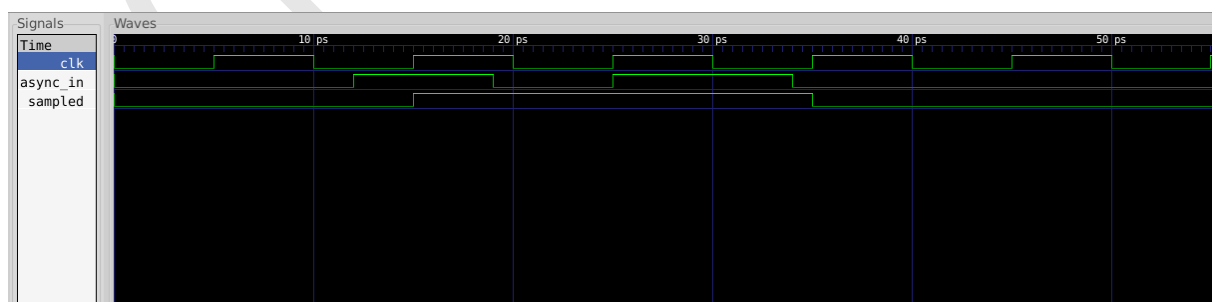
5.1: Execute the simulation script:

```
./run.sh
```

5.2: In GTKWave, add and view the following signals: clk, async\_in, sampled

### Simulation Output

- clk: Regular clock, toggling every 10ns.
- async\_in: Asynchronous input, changes not aligned to clk.
- sampled: The flip-flop output, may exhibit glitches or unpredictable transitions when async\_in changes near a clock edge.



### Observations in GTKWave:

- The sampled output sometimes shows short, spurious pulses or does not settle cleanly when async\_in changes, due to lack of synchronization.
- These glitches represent metastability in simulation.

## Conclusion

Directly sampling asynchronous inputs in digital logic is unsafe and can induce metastability, with unpredictable or glitchy outputs visible in simulation. This experiment visually demonstrates the risks, reinforcing the need for robust synchronization (using chains of flip-flops), which will be addressed in subsequent labs. Careful clock domain crossing and signal synchronization are essential to ensure reliable digital circuit operation.

echiphub.in